

SECTION 1: From Machine Learning to Deep Learning

Recap: Where We've Been

So far, you've learned:

- Supervised Learning (Linear/Logistic Regression, Decision Trees, KNN)
- Unsupervised Learning (K-Means, PCA)
- Classification metrics (Accuracy, Precision, Recall, F1)

All of these worked on **structured/tabular data** — rows and columns, like a spreadsheet.

Why Can't We Use Logistic Regression or Decision Trees on Images?

An image is just a grid of pixel values. A small 224×224 color image has:

$224 \times 224 \times 3$ (RGB channels) = 150,528 individual pixel values

The problem:

- Feeding 150,528 raw numbers into logistic regression would be computationally messy and wouldn't work well
- A single pixel means almost nothing on its own — what matters is the **relationship between neighboring pixels** (edges, curves, textures)
- Traditional ML models have no concept of "spatial structure" — they treat every input as an independent number

This is exactly the gap Deep Learning fills.

SECTION 2: What Is Deep Learning?

Definition: Deep Learning is a subset of Machine Learning that uses **neural networks with many layers** ("deep" = many layers)

to automatically learn patterns from raw data — without manual feature engineering.

Recall from Lesson 1 (AI, ML, DL diagram):

- AI → the broad goal
- ML → subset of AI, learns from data
- DL → subset of ML, uses neural networks, scales well with large data, automates feature extraction

Key difference from classical ML: In classical ML, **you** decide what features matter (e.g., "area" and "rooms" for house price prediction). In Deep Learning, the **network itself learns** which features matter — starting from raw pixels, it discovers edges, then shapes, then whole objects, layer by layer.

SECTION 3: The Neural Network — Core Building Block

The Neuron (Perceptron)

A single artificial neuron does 3 things:

1. Takes inputs ($x_1, x_2, x_3...$)
2. Multiplies each by a weight ($w_1, w_2, w_3...$) and adds a bias (b)
3. Passes the result through an **activation function**

Formula: $z = (w_1x_1 + w_2x_2 + ... + w_nx_n) + b$ output = activation(z)

This should look familiar — it's the **same linear combination formula** from Linear and Logistic Regression! A neural network is essentially many of these stacked together.

Why Activation Functions?

Without an activation function, stacking layers is pointless — multiple linear operations just collapse into one linear operation (no extra learning power).

Activation functions introduce non-linearity, which lets the network learn complex, curved patterns — not just straight lines.

Common activation functions:

Function	Formula	Use
Sigmoid	$1/(1+e^{-z})$	Same one from Logistic Regression! Used for binary output layers
ReLU (Rectified Linear Unit)	$\max(0, z)$	Most common in hidden layers — fast, simple, avoids some training issues
Softmax	$e^{z_i} / \sum e^{z_j}$	Used in output layer for multi-class classification (e.g., which banknote is this?)

Layers of a Neural Network

- **Input layer** — receives the raw data (e.g., pixel values)
- **Hidden layers** — where the actual "learning" and pattern extraction happens
- **Output layer** — produces the final prediction (e.g., probabilities for each class)

"Deep" learning = many hidden layers stacked together.

SECTION 4: How Neural Networks Learn (Recap + Extension)

Just like Linear/Logistic Regression, neural networks learn using:

1. **A loss/cost function** — measures how wrong the prediction is
2. **Backpropagation** — calculates how much each weight contributed to the error (this is really just the **chain rule** applied layer by layer — an extension of the gradient calculation you already did for Logistic Regression)

3. **Gradient Descent** — updates all the weights to reduce the loss, exactly like before, just now across thousands/millions of weights instead of 2-3

Key takeaway for interns: you already understand the core mechanism (cost function + gradient descent) — deep learning just scales this idea up across many more layers and parameters.

SECTION 5: Convolutional Neural Networks (CNNs)

Why CNNs Specifically for Images?

A regular ("fully connected") neural network treats every pixel as a separate independent input — this ignores the fact that **nearby pixels are related** (they form edges, textures, shapes).

CNNs solve this using a special operation: convolution.

The Convolution Operation

Instead of connecting every pixel to every neuron, a CNN slides a small **filter/kernel** (e.g., 3×3 pixels) across the image, looking for specific patterns (like edges, curves, or colors).

Simple analogy: Imagine sliding a small magnifying glass over a photo, checking small patches at a time for a specific pattern (like "is there a vertical edge here?"), and recording the answer at every position.

Key terms:

- **Filter/Kernel** — the small grid of numbers that detects a specific pattern
- **Feature Map** — the output produced after sliding a filter across the image
- **Stride** — how many pixels the filter moves each step
- **Padding** — adding extra border pixels so the output size is controlled

CNN Architecture (Typical Flow)

Input Image

- Convolution Layer (detects basic features: edges, colors)
- Activation (ReLU)
- Pooling Layer (shrinks the image, keeps important info)
- [Repeat Conv + Pool blocks, each layer learning more complex features]
- Flatten (convert 2D feature maps into a 1D vector)
- Fully Connected Layer(s)
- Output Layer (Softmax — final class probabilities)

What each layer type "sees" (intuition):

- Early layers: edges, colors, simple textures
- Middle layers: shapes, patterns, parts of objects (e.g., a note's watermark shape)
- Deep layers: whole objects/concepts (e.g., "this is a 1000 FCFA note")

Pooling Layer (Important Concept)

Pooling shrinks the feature map while keeping the most important information — this reduces computation and helps the model generalize better.

Max Pooling (most common): takes the maximum value from a small region (e.g., 2×2 pixels), discarding the rest.

Why this matters: it makes the model less sensitive to exactly *where* a feature appears in the image (a bit of shift/rotation tolerance).

SECTION 6: Transfer Learning (Critical for Our Project)

The Problem with Training a CNN from Scratch

Training a CNN from scratch requires:

- **Huge amounts of labeled data** (often tens of thousands of images per class)
- **Significant compute power** (GPUs, hours/days of training time)

For an intern project with limited banknote images, this isn't realistic.

The Solution: Transfer Learning

Idea: Use a model that's **already been trained** on a massive dataset (like ImageNet — millions of images, 1000 categories) and **fine-tune** it for our specific task.

Why this works: The early layers of a CNN learn very general features (edges, textures, colors) that are useful for almost *any* image task — not just the original dataset it was trained on. We only need to retrain the final layers to recognize our specific classes (banknote denominations).

Common pretrained models used for transfer learning:

Model	Notes
MobileNetV2	Lightweight, fast, great for mobile/edge deployment — good fit for our project
ResNet50	Deeper, more accurate, heavier computation
EfficientNet	Good balance of accuracy and speed
YOLO	Used for object <i>detection</i> (multiple objects + location) — this is what we'll need in Phase 2

For Phase 1 (single banknote classification), we'll use MobileNetV2 — it's small enough to train quickly even without a powerful GPU, and light enough to eventually deploy on a phone or simple web app.

SECTION 7: Evaluation for Image Classification

Good news — the same metrics you already know still apply here:

- **Accuracy** — overall correct predictions
- **Precision, Recall, F1** — especially useful if some banknote denominations have fewer training images than others (class imbalance, same concept as before)
- **Confusion Matrix** — see exactly which denominations the model confuses with each other (e.g., is the model mixing up 5000 FCFA with 2000 FCFA?)

New addition — Confidence Score: Since CNNs output a Softmax probability for each class, the **confidence** is simply the highest probability value the model assigns.

Example output:

Prediction: 1000 FCFA

Confidence: 94.2%

SECTION 8: Deployment Preview — Gradio

What is Gradio? Gradio is a Python library that lets you turn a trained ML/DL model into a simple, shareable web app — with just a few lines of code. No frontend development knowledge needed.

Why we're using it for this project:

- Upload an image → get a prediction + confidence, displayed in a clean web interface
- Can be hosted for free and shared via a link (great for demos to Traitz clients or during a showcase)

- Works directly with models built in TensorFlow/Keras or PyTorch

Basic Gradio flow (preview — full details in the deployment guide):

```
import gradio as gr
```

```
def predict(image):
```

```
    # preprocess image, run through model, return prediction + confidence
```

```
    ...
```

```
    return prediction, confidence
```

```
demo = gr.Interface(fn=predict, inputs="image", outputs=["text", "text"])
```

```
demo.launch()
```

We'll cover the complete implementation, GitHub setup, and deployment steps in the dedicated deployment guide.

SECTION 9: Summary

- Deep Learning uses neural networks with many layers to automatically learn features from raw data
- Neurons work like a stack of mini logistic regressions, connected by weights and activation functions
- CNNs are built specifically for images — using convolution + pooling to detect patterns regardless of position
- Training from scratch needs massive data; **Transfer Learning** solves this by reusing a pretrained model and fine-tuning it

- We'll use **MobileNetV2** for Phase 1 (single banknote classification)
 - Evaluation still uses Accuracy, Precision, Recall, F1 — plus a new concept: **Confidence Score**
 - Deployment will use **Gradio** to turn the trained model into a usable web app
-

Ready for the Roboflow annotation guide + full Phase 1 deployment steps next — want me to continue straight into that now?